Q1)Dog

Q2)USART is same as UART except sampling done based on a clock signal. Hence the "synchronous". Which is preferred depends on the nature of the peripherals. I would prefer UART cause of its simplicity. In those places where very high speed data transfer is necessary usart can be used.

Q3) It wont be blocking but it also wont recieve anything since it will just return timeout immediately after being called.

Q4)Checked the return value of that HAL_UART_Recieve functio,tracked timeout as a variable and edited it based on return value of that function.

Q5)when it has recieved the byte,rx flag goes high,sending an interrupt. the cpu loads that byte into ram and clears the rx register. cycle continues. When it transmits,once the transmission is done it sets the flag high,sends interrupt. CPU loads the next byte into the transmit register from RAM. Cycle continues.

Q6) Used HAL_GetTick() to keep a sense of time. Updated timeseince-lastreception everytime it recieves something or transmits something. set timetowait based on wether it recieved or transmitted something. After every reception i have to recall the HAL_UART_Receive_IT function.

Q7)In DMA the CPU gives DMA control of the memory to peropheral bus and gives it a starting memory address in ram as well as how many bytes to transfer. From then on dma manages everything on its own,transferring bytes until its done,at which point it sends an interrupt to the CPU. When recieving the CPU tells it how many bytes to recieve and where to put it in RAM. Once DMA has recieved all the relevant bytes it again sends an interrupt to the CPU.

Q8) Same as interrupt method basically.

Q9)Apparently there is a transmit buffer for Serial.print,so if the size of the data to be transmitted isnt too large its non blocking,or else its blocking. Serial,read is non blocking since it just returns -1 if nothing is present.

Q10)Prescaler-¿timer's input clock is divided by this value plus one. its necessary sometimes if our registers are too small and risk overflowing instantly with very high frequency clock signal.Determines PWM frequency.
ARR-¿ Stores the value till which the counter should count before resetting. Determines PWM reolution and frequency.
CCRx-¿If the current counter value is lower than this,outpiuts 1 or else 0. Used to set PWM duty cycle.

$$\text{PWM}_{\text{freq}} = \frac{\text{input frequency}}{(ARR + 1) \times (PSC + 1)}$$

$$\text{PWM}_{\text{duty cycle}} = \frac{\text{CCR}}{\text{ARR} + 1}$$

Q11)Most of the things came prebuilt from MX. just had to initialize the PWM by typing HAL_TIM_PWM_Start(&htim1,TIM_CHANNEL_1);